



Bangalore University
Project Report On

SWIFT RIDES USING DJANGO FRAMEWORK

*Submitted in partial fulfillment of the requirement for the award of degree in
Bachelor of Computer Applications during the Academic year 2023-24*

Submitted by
Chethan KR – U03KU21S0066

Under the guidance of
Asst. Professor Supriya D



SOUNDARYA
INSTITUTE OF
MANAGEMENT &
SCIENCE



**SOUNDARYA
INSTITUTE OF
MANAGEMENT &
SCIENCE**

DEPARTMENT OF COMPUTER SCIENCE

Certificate

This is to certify that **Chethan KR** bearing Reg. No **U03KU21S0066** has satisfactorily completed the project entitled **SWIFT RIDES USING DJANGO FRAMEWORK** under the guidance and supervision of **Supriya D** Asst. Professor, Department of Computer Science, in partial fulfillment of VI Semester BCA course as prescribed by Bangalore University, Bangalore in our college during the academic year 2023-24

Project Guide

Asst. Prof. Supriya D

Head of the Department

Examiners

1.....

2.....

DECLARATION OF GUIDE

The Project work entitled **SWIFT RIDES USING DJANGO FRAMEWORK** is based on work carried out by **Chethan KR** during the course of study under the supervision of **Asst. Professor Supriya D.** I assert the statements made and conclusions drawn are the outcome of Project work and hereby approve for the submission for the partial fulfillment for the award of Bachelor of Computer Applications under Bangalore University for the year 2023-24.

Asst. Prof. SUPRIYA D

ACKNOWLEDGEMENT

I articulate my sincere gratitude to all those who helped me in making this venture a grand success, without whose constructive criticism as well as words of inspiration this project of mine would not have seen the light.

I take this opportunity to thank **Dr. Vasu B.A** Principal, Soundarya Institute Of Management & Science, Bangalore for permitting me to undertake this project as part of my curriculum.

I would like to mention my sincere gratitude to **Prof. Vijay Kumar AS**, Head of the Department and my guide **Asst. Supriya D** for the guidance and encouragement that has been a source of inspiration for me and also for giving me the valuable information and advices that helped me a lot for the successful completion of my project work.

Last but not the least, I would like to express my sincere thanks to my dear parents and friends for accompanying us throughout the completion of the project.

THANK YOU.

Chethan KR

Index

Chapters.	Details
Chapter - 1	• Introduction • Statement of the problem • Objective of the study • Scope of the study • Feasibility study * Technical Feasibility * Operational Feasibility * Financial Feasibility
Chapter - 2	2.1 Existing System 2.2 Limitations of Existing System 2.3 Proposed System 2.4 Advantages of Proposed System 2.5 Problem Formulation 2.6 Methodology
Chapter - 3	3.1 Hardware & Software requirement 3.2 Front End 3.3 Back End.
Chapter – 4	4.1 Structured chart 4.2 UML Diagram 4.3 Data Flow diagram 4.4 Data base design
Chapter - 5	5.1 Coding 5.2 Output screen 5.3 Testing
Chapter - 6	6.1 Conclusion 6.2 Future Enhancement 6.3 References

Chapter-1

Introduction

- Introduction
- Statement of the problem
- Objective of the study
- Scope of the study
- Feasibility study
 - * Technical Feasibility
 - * Operational Feasibility
 - * Financial Feasibility

1.1 Introduction

Full Stack Development refers to the practice of working on both the front end (client side) and the back end (server side) of a web application. A Full Stack Developer is proficient in all layers of web development, from the user interface to the database, server, and everything in between. This versatility allows them to build and manage an entire web application from start to finish.

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. Created by experienced developers, it takes care of much of the hassle of web development, so developers can focus on writing their applications without needing to reinvent the wheel.

1.2 Statement of the problem:

The car rental industry has seen a significant shift towards online platforms, making it easier for customers to book vehicles from the comfort of their homes. Despite this progress, many existing platforms lack user-friendly interfaces, efficient booking processes, and robust management systems for fleet operators. There is a need for a comprehensive web application that addresses these gaps and provides a seamless experience for both customers and car rental businesses.

1.3 Objective of the Study

To develop a Swift Rides that allows users to easily search for, book, and manage car rentals while providing rental companies with tools to manage their fleet, track reservations, and handle customer interactions efficiently.

1.4 Scope of the study

The economic feasibility of the Swift Rides project evaluates whether the benefits outweigh the costs. This involves estimating development costs (including salaries, tools, and infrastructure), ongoing operational expenses (such as hosting and maintenance), and potential revenue from user bookings and possibly subscription models for rental companies. The project is economically feasible if the expected revenue and cost savings (e.g., streamlined operations for rental companies) exceed the total costs. A positive return on investment (ROI) and break-even analysis would indicate financial viability, making the project a sound economic decision.

1.5 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ◆ TECHNICAL FEASIBILITY
- ◆ OPERATIONAL FEASIBILITY
- ◆ FINANCIAL FEASIBILITY

Technical Feasibility

The technical feasibility of Swift Rides is highly promising, given the availability of robust web development frameworks, secure payment gateways, and scalable cloud infrastructure. Using Django for backend development ensures a reliable and efficient server-side architecture, while front-end technologies like HTML, CSS, JavaScript and Tailwind CSS provide a responsive and intuitive user experience. Automated testing tools and CI/CD pipelines enhance development efficiency and code quality. Moreover, modern data security practices and compliance with regulations ensure the protection of user data, making the technical implementation of Swift Rides both viable and sustainable.

Operational Feasibility

The operational feasibility of Swift Rides is strong, as it leverages existing technologies and industry practices to streamline car rental operations. The platform can be managed by a team with expertise in web development, customer service, and fleet management. Automated systems for booking, payment processing, and vehicle tracking reduce manual intervention, enhancing efficiency and accuracy. Integration with third-party services for payments and insurance simplifies operations, while real-time data and analytics support informed decision-making. Additionally, user-friendly interfaces and mobile compatibility ensure accessibility and convenience for customers, making the day-to-day operations of Swift Rides manageable and scalable.

Financial Feasibility

The financial feasibility of Swift Rides appears promising, with potential for high returns on investment through various revenue streams such as rental fees, service charges, and premium add-ons. Initial costs include development, marketing, and integration with third-party services, which can be managed through a phased approach to minimize upfront expenses. The scalability of the platform allows for gradual expansion based on market demand, reducing financial risk. Additionally, leveraging cloud services reduces infrastructure costs, while automated systems cut down on operational expenses. A strong business model focusing on customer satisfaction and retention can drive steady income, making the financial outlook for Swift Rides favorable.

Chapter -2

System Study

- 2.1 Problem Statement
- 2.2 Existing System
- 2.3 Limitations of Existing System
- 2.4 Proposed System
- 2.5 Advantages of Proposed System
- 2.6 Methodology

2.1 Problem Statement:

Swift Rides is a car rental web application that aims to provide users with a seamless and efficient platform to rent vehicles online. The application allows users to browse a catalog of available cars, filter their search based on various criteria such as car type, price, rental period, and location, and view detailed information about each vehicle, including photos, specifications, and rental terms. Users can create accounts, manage their profiles, and track their rental history. The application supports secure online payments, provides booking confirmations, and sends notifications about the status of rentals.

Additionally, Swift Rides offers an administrative interface for managing vehicle listings, pricing, availability, and reservations. The administrative tools include functionalities for managing user accounts, handling customer inquiries and support tickets, and generating reports on rental activities and revenue. The platform integrates with third-party services such as payment gateways, insurance providers, and GPS tracking for enhanced functionality.

To enhance user experience, Swift Rides includes features such as personalized recommendations based on user preferences and rental history, loyalty programs offering discounts or rewards for frequent rentals, and options for adding extras like child seats or GPS navigation systems to the rental.

The primary challenge is to develop a user-friendly interface that ensures a smooth booking process, integrates secure payment gateways, and maintains an up-to-date inventory of available vehicles while ensuring data security and privacy for users. The system is designed to be scalable to handle peak usage times, with robust backup and recovery processes in place, and compliance with relevant legal and regulatory requirements related to car rentals and data protection.

2.2 Existing System

There are several existing systems similar to Swift Rides in the car rental industry. Here are a few notable examples:

1. **Turo:** Turo is a peer-to-peer car sharing platform that allows individuals to rent out their personal vehicles to others. It operates similarly to a rental service but with a focus on community sharing.
2. **Enterprise Rent-A-Car:** Enterprise is a traditional car rental company that offers a wide range of vehicles for short-term rentals. It has a strong presence in various locations and provides both online and offline booking services.
3. **Hertz:** Hertz is another well-known car rental company that operates globally. It offers a variety of rental options, including standard cars, SUVs, and luxury vehicles, with an emphasis on customer service and loyalty programs.
4. **Zipcar:** Zipcar is a membership-based car sharing service that provides access to vehicles on a short-term basis. It's popular in urban areas and on college campuses, offering flexibility and convenience for users who don't own a car.
5. **Avis Budget Group:** Avis and Budget are two brands under the Avis Budget Group that provide car rental services worldwide. They cater to both leisure and business travelers with a range of vehicle options and rental plans.

These systems vary in their business models, target audiences, and service offerings but share the common goal of providing convenient and reliable car rental solutions to consumers. Each platform may have unique features such as loyalty programs, mobile app integrations, and specialized rental categories to differentiate themselves in the market.

2.3 Limitations of Existing System

While car rental systems like Turo, Enterprise, Hertz, Zipcar, and Avis Budget Group offer convenient solutions for renting vehicles, they also come with certain limitations:

- 1.**Availability and Location:** Availability of vehicles can be limited, especially during peak seasons or in certain locations. This may lead to challenges in finding the desired vehicle type or availability at preferred pickup locations.
- 2.**Booking Flexibility:** Some systems may have strict booking policies, including minimum rental periods or advance booking requirements, which can restrict flexibility for last-minute rentals or short-term needs.
- 3.**Cost and Fees:** Rental costs can vary significantly based on factors such as vehicle type, rental duration, location, and additional fees (e.g., insurance, fuel charges, mileage limits). These costs may not always be transparent upfront, leading to unexpected expenses.
- 4.**Insurance and Liability:** Insurance coverage policies may vary, and renters may need to navigate complex terms and conditions regarding liability, damages, and coverage limitations. This can lead to confusion and uncertainty about financial responsibilities in case of accidents or damages.
- 5.**Customer Support:** Depending on the system, customer support quality and responsiveness may vary. Issues such as vehicle maintenance, billing discrepancies, or booking modifications may not always be resolved promptly or satisfactorily.
- 6.**User Experience and Interface:** Some systems may have outdated or cumbersome user interfaces, making it challenging for users to navigate and complete bookings efficiently. Mobile app usability and functionality may also vary, affecting overall user experience.
- 7.**Vehicle Condition and Maintenance:** Ensuring the reliability and cleanliness of rental vehicles can be a concern. Systems that do not prioritize regular maintenance or vehicle inspection processes may lead to dissatisfaction among renters.
- 8.**Environmental Impact:** Traditional car rental systems often involve large fleets of vehicles, contributing to environmental concerns related to fuel consumption, emissions,

and overall sustainability practices.

9.Regulatory Compliance: Compliance with local regulations, licensing requirements, and consumer protection laws can vary across regions where these systems operate. This may affect service availability, pricing structures, and operational practices.

10. Competition and Market Dynamics: Intense competition in the car rental industry can lead to pricing pressures and promotional tactics that may not always benefit consumers. Limited differentiation in service offerings may also restrict choices for renters.

Addressing these limitations requires ongoing improvements in technology, customer service standards, transparency in pricing and policies, and commitment to sustainability and regulatory compliance within the car rental industry.

2.4 Proposed System

- **User-friendly Interface:** Develop a responsive web application with an intuitive interface for easy navigation and seamless user experience.
- **Comprehensive Vehicle Catalog:** Create a detailed catalog featuring various vehicle types, complete with high-quality images, specifications, and rental terms displayed clearly.
- **Advanced Search and Filtering:** Implement robust search and filtering functionalities to allow users to easily find vehicles based on criteria such as type, price range, location, and availability.
- **User Account Management:** Enable users to create accounts, manage profiles, store preferences, and track rental history securely.
- **Secure Payment Gateway Integration:** Integrate reliable and secure payment gateways to facilitate seamless online transactions, including support for multiple payment methods and currencies.
- **Real-time Availability Updates:** Implement real-time updates to ensure accurate availability status of vehicles, reducing instances of overbooking and enhancing user satisfaction.
- **Booking Management:** Provide a streamlined booking process with options for flexible rental durations, instant booking confirmations, and automated notifications.
- **Administrative Dashboard:** Develop a comprehensive dashboard for administrators to manage vehicle listings, pricing, availability, customer inquiries, and support tickets efficiently.
- **Customer Support Features:** Implement features for handling customer inquiries, feedback, and support requests promptly, ensuring high-quality customer service.
- **Customizable Booking Options:** Offer customizable booking options, including add-ons such as insurance coverage, GPS navigation systems, and child seats, to meet diverse customer needs.
- **Performance Analytics and Reporting:** Incorporate analytics tools to generate insights into rental activities, revenue trends, user behaviors, and operational performance.

- **Mobile Compatibility:** Ensure compatibility with mobile devices, offering a responsive design and mobile app integration for convenient on-the-go bookings and management.
- **Integration with External Services:** Integrate with third-party services such as insurance providers, roadside assistance, and GPS tracking systems to enhance service offerings.
- **Scalability and Reliability:** Design the system to be scalable to handle peak loads and reliable to minimize downtime, supported by robust backup and recovery mechanisms.
- **Data Security and Privacy:** Implement stringent data security measures to protect user information, comply with data protection regulations, and ensure secure handling of sensitive data.
- **Continuous Improvement and Updates:** Commit to ongoing system improvements, feature enhancements, and updates based on user feedback, industry trends, and technological advancements.

2.5 Advantages of proposed System

- Enhanced user experience
- Comprehensive vehicle information
- Secure and convenient payments
- Real-time availability updates
- Efficient operations management
- Responsive customer support
- Customizable booking options
- Mobile compatibility
- Integration with third-party services
- Scalability and reliability
- Data security and privacy
- Localization and internationalization support
- Continuous improvement through updates

2.6 Methodologies

The methodology for developing Swift Rides adopts an agile approach, starting with gathering requirements and designing user interfaces and system architecture. Development progresses iteratively using technologies like HTML/CSS/JavaScript for front-end and Django/Node.js for back-end. Git manages version control, enabling collaborative development. Continuous integration and deployment

(CI/CD) automates testing and deployment. Testing includes unit and integration tests, with user acceptance testing (UAT) for final validation. Deployment to production involves setting up monitoring and logging. Post-deployment, agile practices drive ongoing maintenance and updates based on user feedback and market needs, ensuring Swift Rides evolves efficiently and meets user expectations.

Chapter -3

System Requirement

3.1 Hardware & Software requirement

3.2 Front End

3.3 Back End.

3.4 Database

3.1 Hardware Software Requirements

HARDWARE REQUIREMENTS:

- Processor : Intel Core i3-3110M
- Storage : Minimum of 10 GB.
- RAM : Minimum of 4 GB

SOFTWARE REQUIREMENTS:

- Operating system : Windows 11/10.
- Coding Language : Python
- Web Framework : Django
- Software/IDE : Visual Studio Code

Front End:

- HTML
- CSS
- Java Script
- Tailwind CSS

Back End:

- Python
- Django

Database:

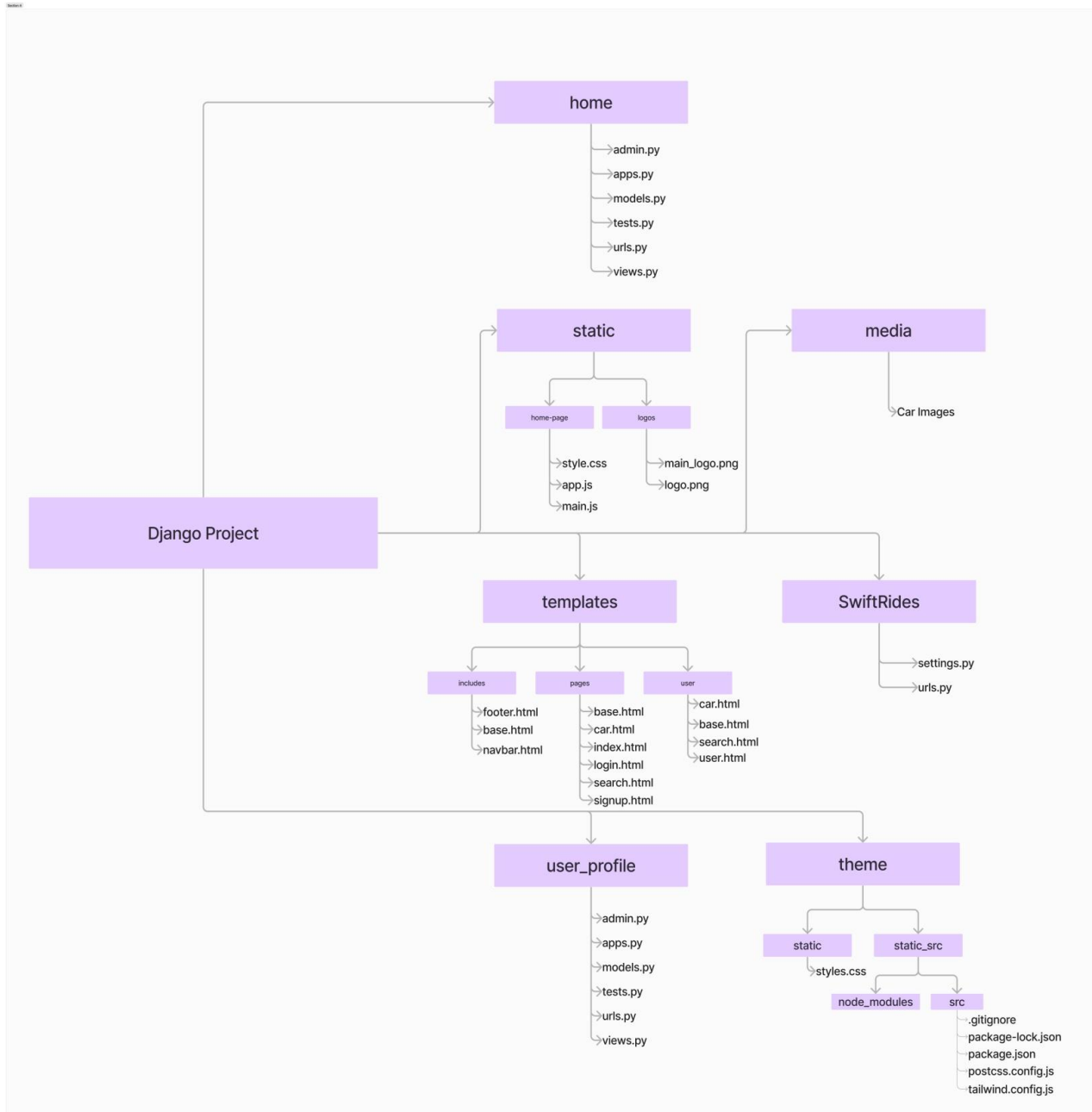
- SQLite

Chapter -4

System Design

- 4.1 Structured chart.
- 4.2 ER Diagram.
- 4.3 Data Flow diagram.
- 4.4 Data base design.

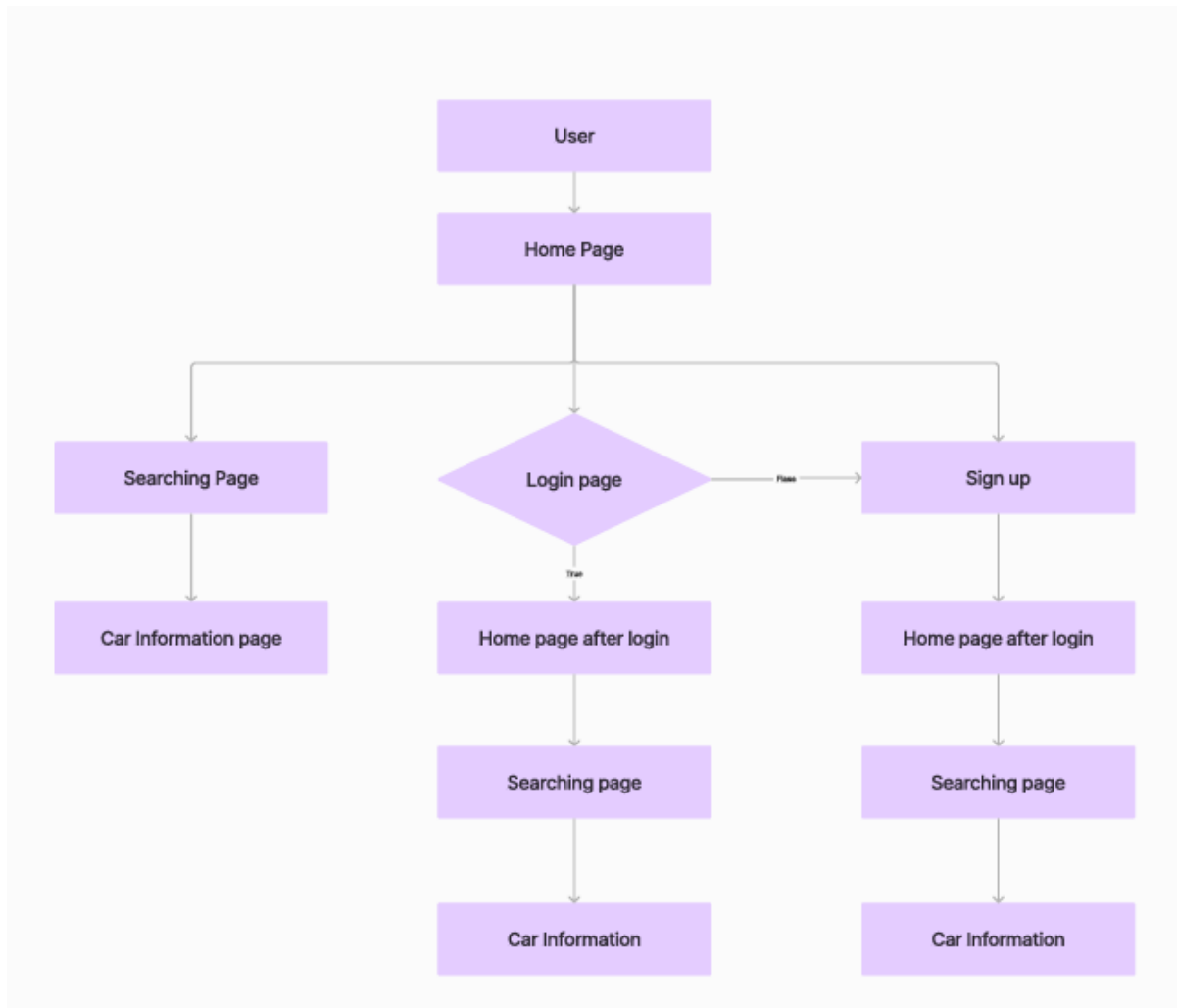
4.1 Structured chart



4.2 ER DIAGRAM:



4.3 Data Flow Diagram



4.4 Database design

db.sqlite3 ▶ **home_location**

Reset Filters Records: 13

	id	city	address
	Search column...	Search column...	Search column...
1	1	Bangalore	market
2	2	Mumbai	Mumbai
3	3	Delhi	Delhi
4	4	Chennai	Chennai
5	5	Hyderabad	Hyderabad
6	6	Pune	Maharashtra
7	7	Jaipur	Jaipur
8	8	Lucknow	Lucknow
9	9	Nagpur	Nagpur
10	10	Kota	Kota
11	11	Mysuru	Mysuru
12	12	Tumkuru	Tumkuru
13	13	Sira	Sira(madakasira)

db.sqlite3 ▶ **home_car**

Reset Filters Records: 7

	id	make	model	year	color	mileage	price_per_day	available
	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...
1	2	Ford	Bronco AMT	2021	magma grey	15	6999	1
2	3	Jeep	Compass	2021	black	18	7999	1
3	4	Maruti Suzuki	Ignis (NEXA)	2023	Black (dark edition)	22	2999	1
4	5	Nissan	Sentra	2016	stardust brown	18	3999	1
5	6	Tata Motors	Tiago	2024	funky Orange	24	1999	1
6	7	Toyota	Corolla	2020	White	22	1998	1
7	8	Volkswagen	Golf MK4	2019	Space blue	16	6999	1

Chapter -5

System Implementation

5.1 Coding

5.2 Output screen

5.3 Testing

5.1 Code

#Project folder urls.py

```
from django.contrib import admin
from django.urls import path, include
from home import views
from user_profile import views as user_views
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('__reload__/', include('django_browser_reload.urls')),

    path("", include('home.urls')),
    path('search/', views.searching, name='search'),

    path('user/', include('user_profile.urls')),
    path('UserLog/', user_views.UserLog, name='UserLog'),
    path('user_search/', user_views.user_searching, name='user_searching'),

    path('login/', user_views.Login, name='login'),
    path('signup/', user_views.Signup, name='signup'),
    path('logout/', user_views.Logout, name='logout'),

    path('car/<int:car_id>', views.car_detail, name='car_details'),
    path('car/make/<str:make>', views.car_detail_by_make,
        name='car_detail_by_make'),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

#Home urls.py

```
from django.urls import path
from home import views
urlpatterns = [
    path("", views.index, name='home'),
]
```

#Home views.py

```
from django.http import HttpResponseRedirect
from django.shortcuts import render, get_object_or_404
from .models import Car, Location
# Create your views here.
def index(request):
    return render(request, 'pages/index.html')
def searching(request):
    query = request.GET.get('search')
    destination = request.GET.get('destination')
    start_date = request.GET.get('start_date')
    start_datetime = request.GET.get('start_datetime')
    end_date = request.GET.get('end_date')
    end_datetime = request.GET.get('end_datetime')
    results = Car.objects.all()
    if query:
        results = results.filter(locations__city__icontains=query)
    return render(request, 'pages/search.html', {'results': results, 'query': query, 'destination': destination})
    # return HttpResponseRedirect('this is search page')
def car_detail(request, car_id):
    car = get_object_or_404(Car, pk=car_id)
    return render(request, 'pages/car.html', {'car': car})
```

```
def car_detail_by_make(request, make):
    try:
        car = Car.objects.get(make=make)
        return render(request, 'pages/car.html', {'car': car})
    except Car.DoesNotExist:
        return HttpResponseNotFound("Car with make '{}' does not exist.".format(make))
```

#User views.py

```
from django.shortcuts import render, HttpResponseRedirect ,redirect
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from django.contrib.auth import login , logout , authenticate
from django.contrib import messages
from django.db import IntegrityError
from django.contrib.auth.decorators import login_required
from django.http import HttpResponseNotFound
from django.shortcuts import render, HttpResponseRedirect, get_object_or_404
from home.models import Car, Location
from django.core.validators import validate_email
from django.core.exceptions import ValidationError
import re
from django.views.decorators.cache import cache_control
# Create your views here.
def Login(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
```

```
    return redirect('UserLog') # Redirect to some logged-in page
else:
    messages.error(request, 'Invalid username or password.') # Use error message
    return redirect('login') # Redirect back to login page
else:
    return render(request, 'pages/Login.html') # Render the login page template

@cache_control(no_cache=False, must_revalidate=False, no_store=False)
@login_required(login_url='login')
def UserInfo(request):
    return render(request, 'user/UserInformation.html')

@cache_control(no_cache=False, must_revalidate=False, no_store=False)
@login_required(login_url='login')
def UserLog(request):
    return render(request, 'user/user.html')

def Signup(request):
    if request.method == 'GET':
        return render(request, 'pages/Signup.html', {"form": UserCreationForm()})
    else:
        username = request.POST['username']
        password1 = request.POST['password1']
        password2 = request.POST['password2']

        # Username validation
        if User.objects.filter(username=username).exists():
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Username
already taken'})
        if len(username) < 4:
```

```
        return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error':
'Username must be at least 4 characters long'})
        if not re.match(r'^[a-zA-Z0-9_]+$', username):
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Username can
only
contain letters, numbers, and underscores'})
        # Check password match
        if password1 != password2:
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Passwords do
not match'})
        # Password validation
        if len(password1) < 8:
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Password must
be at least 8 characters long'})
        if not re.search(r"[A-Z]", password1):
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Password must
contain at least one uppercase letter'})
        if not re.search(r"[a-z]", password1):
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Password must
contain at least one lowercase letter'})
        if not re.search(r"[0-9]", password1):
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Password must
contain at least one digit'})
        if not re.search(r"[\W_]", password1):
            return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'Password must
contain at least one special character'})
        try:
            user = User.objects.create_user(username, password=password1)
            user.save()
```

```
    login(request, user)
    return redirect('UserLog')
except IntegrityError:
    return render(request, 'pages/Signup.html', {'form': UserCreationForm(), 'error': 'An error
occurred during signup. Please try again.'})

def Logout(request):
    logout(request)
    return redirect('login')

def user_searching(request):
    query = request.GET.get('search')
    destination = request.GET.get('destination')
    start_date = request.GET.get('start_date')
    start_datetime = request.GET.get('start_datetime')
    end_date = request.GET.get('end_date')
    end_datetime = request.GET.get('end_datetime')
    results = Car.objects.all()
    if query:
        results = results.filter(locations__city__icontains=query)

    return render(request, 'user\search.html', {'results': results, 'query': query, 'destination': destination})

# return HttpResponse('this is search page')
```


#Home models.py

```
from django.db import models
from django.contrib.auth.models import User
import os

# Create your models here.
class Location(models.Model):
    city = models.CharField(max_length=100)
    address = models.CharField(max_length=255)
    # Other relevant location information

    def __str__(self):
        return self.city

def car_image_path(instance, filename):
    # Get the make, model, and ID of the car
    make = instance.make
    model = instance.model
    car_id = instance.id
    # Extract the image type from the field name
    # Assuming the filename format is '<type>_<original_filename>'
    image_type = filename.split('_')[0]
    # Generate the folder path for the specific car
    car_folder = f'{make}_{model}_car_{car_id}'
    # Generate the filename using make, model, car ID, image type, and the original filename
    if image_type == 'car_image_primary':
        image_order = '1'
    elif image_type == 'car_image_secondary1':
        image_order = '2'
    elif image_type == 'car_image_secondary2':
        image_order = '3'
```

else:

 # Handle other image types if needed

 image_order = 'other'

 filename = f"{make}_{model}_{car_id}_{image_order}_{filename}"

 # Return the path where the file will be uploaded

 return os.path.join('cars', 'images', car_folder, filename)

class Car(models.Model):

 id = models.AutoField(primary_key=True)

 make = models.CharField(max_length=100)

 model = models.CharField(max_length=100)

 year = models.IntegerField()

 color = models.CharField(max_length=50)

 mileage = models.DecimalField(max_digits=10, decimal_places=2)

 price_per_day = models.DecimalField(max_digits=10, decimal_places=2)

 description = models.TextField(blank=True)

 available = models.BooleanField(default=True)

 car_image_primary = models.ImageField(upload_to=car_image_path, default="")

 car_image_secondary1 = models.ImageField(

 upload_to=car_image_path, blank=True)

 car_image_secondary2 = models.ImageField(

 upload_to=car_image_path, blank=True)

 locations = models.ManyToManyField(Location, related_name='cars')

 def __str__(self):

 return f"{self.year} {self.make} {self.model}"

#Home admin.py

```
from django.contrib import admin
# Register your models here.
from . models import Location, Car, RentalTransaction
admin.site.register(Location)
admin.site.register(Car)
admin.site.register(RentalTransaction)
```

#navbar.html

```
{% extends 'includes/include_base.html' %}
{% load static tailwind_tags %}
{% load static %}
{% block links %}
{% tailwind_css %}

{% endblock %}
{% block content %}
<nav class=" sm:flex sm:justify-center sm:items-center ">
  <div class="top-nav w-full flex justify-between items-start ">
    <div class="logo ">
      <a href="{% url 'home' %}"></a></div>
      <div class="menu-i text-[2rem] relative top-[0.5rem] mr-4 sm:hidden ">
        <ion-icon name="menu" id="menu"></ion-icon>
      </div>
    </div>
    <div class="nav-btt ">
      <div class="nav-list">
        <div class="hidden sm:block ">
```

```

<ul class="text-[1.11rem] bg-white text-[#1c1b1b] absolute left-0 top-
[4rem] h-[100vh] w-full z-50 ||
    sm:relative sm:left-0 sm:top-0 sm:bg-transparent sm:text-[0.8rem] sm:h-
full sm:w-[30rem] sm:gap-[1rem] sm:flex sm:justify-between
    sm:translate-x-0
    md:relative md:left-0 md:w-[50rem] md:text-[1rem] md:justify-end
">

```

```

<hr class="my-4">
<li class=" py-3 ml-4 hover:text-[#9960fc] "><a href="{% url 'home'
%}">Home</a></li>
<li class=" py-3 ml-4 hover:text-[#9960fc] "><a href="{% url 'search'
%}">Search</a></li>
<hr class="my-4 sm:hidden">
<li class=" py-3 ml-4 hover:text-[#9960fc] ">Become a host</li>
<!-- error in login and sign up, when i use url 'Login' and url 'Signup' -->
<li class=" py-3 ml-4 hover:text-[#9960fc] "><a href="{% url 'login' %}">Log
in</a></li>
<li class=" py-3 ml-4 sm:mr-[3rem] sm:ml-0 hover:text-[#9960fc] "><a
href="{% url 'signup' %}">Sign
up</a></li>
<hr class="my-4 sm:hidden">
<li class=" py-3 ml-4 hover:text-[#9960fc] sm:hidden"><a
href="#footer">FAQ</a></li>
<li class=" py-3 ml-4 hover:text-[#9960fc] sm:hidden"><a
href="#footer">Terms of
    Services</a>
</li>
</ul>
</div>

```

</div>

</div>

</nav>

<hr>

{% endblock %}

#base.html

{% load static tailwind_tags %}

{% load static %}

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>{% block title %} {% endblock %}</title>

<!-- the linked in external css and script in static -->

{% block links %}

<link rel="stylesheet" href="{% static 'home-page/style/style.css' %}">

<script src="{% static 'home-page/script_for_home/home.js' %}"></script>

<script src="{% static 'home-page/script_for_home/app.js' %}"></script>

<link rel="shortcut icon" href="{% static 'logos/swift rides.png' %}" >

{% endblock %}

<!-- the external links for font , icon , -->

<script type="module"

src="https://unpkg.com/ionicons@7.1.0/dist/ionicons/ionicons.esm.js"></script>

<script nomodule

src="https://unpkg.com/ionicons@7.1.0/dist/ionicons/ionicons.js"></script>

<link rel="preconnect" href="https://fonts.googleapis.com">

<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

```
<link
href="https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,400..900;1,400.
.9
00&display=swap" rel="stylesheet">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Geologica:wght@100..900&display=swap"
rel="stylesheet">
  <script src="https://kit.fontawesome.com/5d4cb69663.js"
crossorigin="anonymous"></script>
  {% tailwind_css %}
</head>
<body
  class="max-w-[350px] m-auto | sm:bg-red-00 sm:max-w-[750px] | md:bg-blue-00 md:max-
w-[1010px] | lg:bg-green-00 lg:max-w[1270px] | xl:bg-purple-00 xl:max-w-[1530px] | 2xl:bg-
white overflow-x-hidden ">
  {% include 'includes/navbar.html' %}
  {% block main %}
  {% endblock %}
  {% include 'includes/footer.html' %}
</body>
</html>
```

#index.html

```
{% extends 'pages/basic.html' %}
{% load static tailwind_tags %}
{% block title %} SwiftRides {% endblock %}
{% block links %}
{% load static %}
<link rel="stylesheet" href="{% static 'home-page/style/style.css' %}">
<script src="{% static 'home-page/script_for_home/home.js' %}"></script>
<script src="{% static 'home-page/script_for_home/app.js' %}"></script>
<link rel="shortcut icon" href="{% static 'logos/mainlogo.jpg' %}" type="image/x-icon">
{% endblock %}
{% block main %}
<main class="select-none">
  <section id="hero">
    <div class="searching  ">
      <div class="form bg-white absolute top-24 ml-4 rounded-lg w-[20rem] z-10 sm:top-[6rem]
sm:max-[750px] sm:w-[90vw] sm:ml-[2rem] sm:rounded-full md:top-[7rem] lg:w-[75vw] xl:w-
[75rem] xl:ml-[5rem] xl:translate-x-[1.5rem] xl:translate-y-[1rem]">
        <form action="/search" method="get" id="searchForm"
          class="relative m-2 sm:flex sm:text-center lg:w-[70vw] xl:w-[90vw] xl:left-[3rem] xl:top-
[0.4rem]">
          <label for="address" class="absolute text-[0.9rem] sm:ml-2 md:text-
[1rem]">Where</label>
          <input name="search" id="search" type="search" class="address w-full rounded-md
border-0 pt-4 text-gray-900 focus:ring-2 focus:ring-inset focus:ring-transparent focus:border-
none sm:w-[10rem] sm:h-[2rem] sm:mt-3 sm:ml-2 sm:mr-0 outline-none md:w-[10rem] md:text-
[1.1rem] xl:w-[15rem] xl:h-[2.5rem] " placeholder="City,address or hotel"> <br>
          <label for="date_from"
            class="absolute sm:translate-x-[11.4rem] text-[0.9rem] md:text-[1rem] lg:translate-x-
[11.5rem] xl:translate-x-[16.5rem]">From</label>
```

```

<input type="date" name="start_date" id="date_from" class="h-8 w-[9rem] my-4 rounded-md pt-4
text-gray-900 placeholder:text-gray-400 focus:ring-inset focus:ring-transparent focus:border-none
border-none outline-none p-[14px] translate-y-1 sm:w-[15rem] sm:h-[2rem] sm:mt-4 md:text-
[1.2rem] xl:w-[12rem] ">

    <input type="time" name="start_datetime" id="time_from" class="h-8 w-[9rem] my-4
rounded-md p-[14px] translate-y-1 pt-4 text-gray-900 placeholder:text-gray-400 focus:ring-inset
focus:ring-transparent focus:border-none border-none outline-none sm:w-[15rem] sm:h-[2rem] sm:mt-
4 md:text-[1.2rem] md:p-[1rem] xl:w-[12rem] ">

    <br>

    <label for="date_until"
        class="absolute sm:translate-x-[25.5rem] text-[0.9rem] md:text-[1rem] md:translate-x-
[32rem] lg:translate-x-[30rem] xl:translate-x-[40.5rem]">Until</label>

    <input type="date" name="end_date" id="date_until" class="h-8 w-[9rem] my-4 rounded-
md p-[14px] translate-y-1 pt-4 text-gray-900 focus:ring-inset focus:ring-transparent focus:border-none
border-none outline-none sm:w-[15rem] sm:h-[2rem] sm:mt-4 md:text-[1.2rem] xl:w-[12rem]">
<input type="time" name="end_datetime" id="time_until" class="h-8 w-[9rem] my-4 rounded-md p-
[14px] translate-y-1 pt-4 text-gray-900 focus:ring-inset focus:ring-transparent focus:border-none
border-none outline-none sm:w-[15rem] sm:h-[2rem] sm:mt-4 md:text-[1.2rem] md:p-[1rem] xl:w-
[12rem] ">

    <br>

    <hr>

    <div class="btt w-full text-center my-2 sm:w-14 sm:pl-4 ">

        <button type="submit" class="w-full h-10 text-6 text-white font-bold bg-[#9960fc]
hover:bg-transparent hover:border-solid hover:border-2 hover:border [#9960fc] hover:text-[#9960fc]
sm:w-10 sm:rounded-full xl:mr-10 xl:translate-x-[2rem] ">

            <span class="block sm:hidden ">Search for cars</span>

            <span class="hidden sm:block sm:text-[1.3rem] sm:pt-1"><ion-icon name="search"
id="search"></ion-icon></span>

        </button>

    </div>

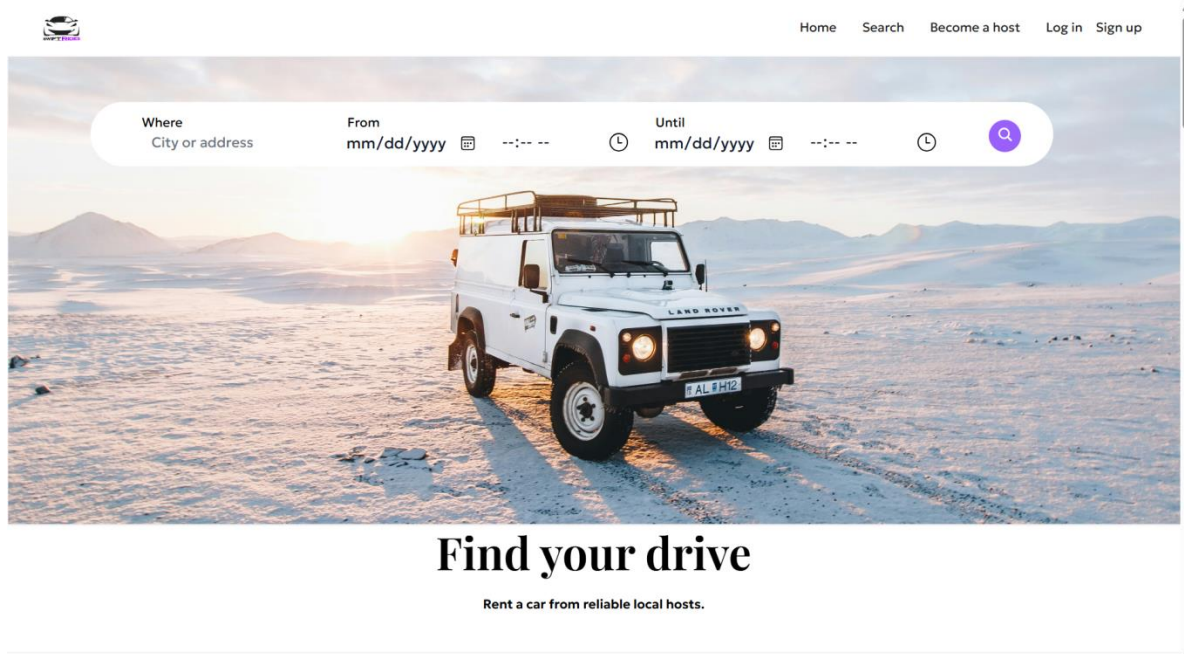
```



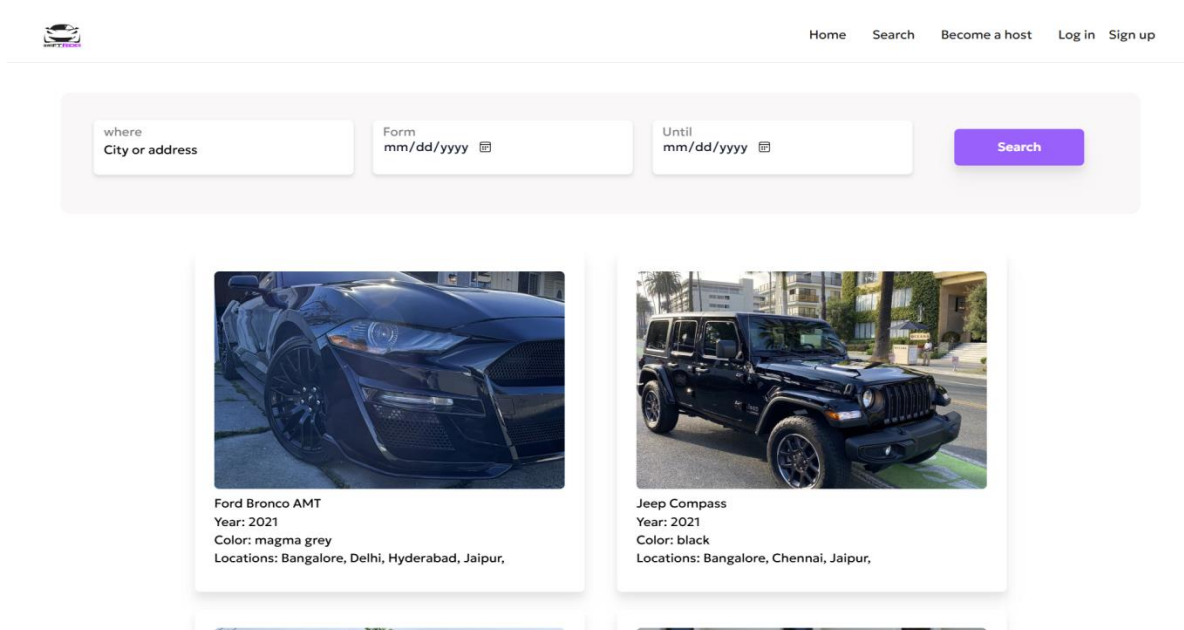
```
</form>
</div>
<div class="hero ">
  <div class="hero-img relative">
    
  </div>
  <div class="hero-txt w-full text-center mt-[-1rem] ">
    <h1 class="text-[2.5rem] mt-4 sm:text-[3.5rem] font-[650] sm:mt-2 google_font">Find
your drive</h1>
    <p class="text-[1rem] font-semibold mt-5 sm:m-3 ">Rent a car from reliable local
hosts.</p>
  </div>
</div>
</div>
</section>
</body>
</html>
```

5.2 Output Screen

Home page:



Search page:



Login page:

Home Search Become a host Log in Sign up

Welcome to SwiftRides

Log in

[Forgot password?](#)

Login

Don't have an account? [Sign up](#)

Sign up page:

Home Search Become a host Log in Sign up

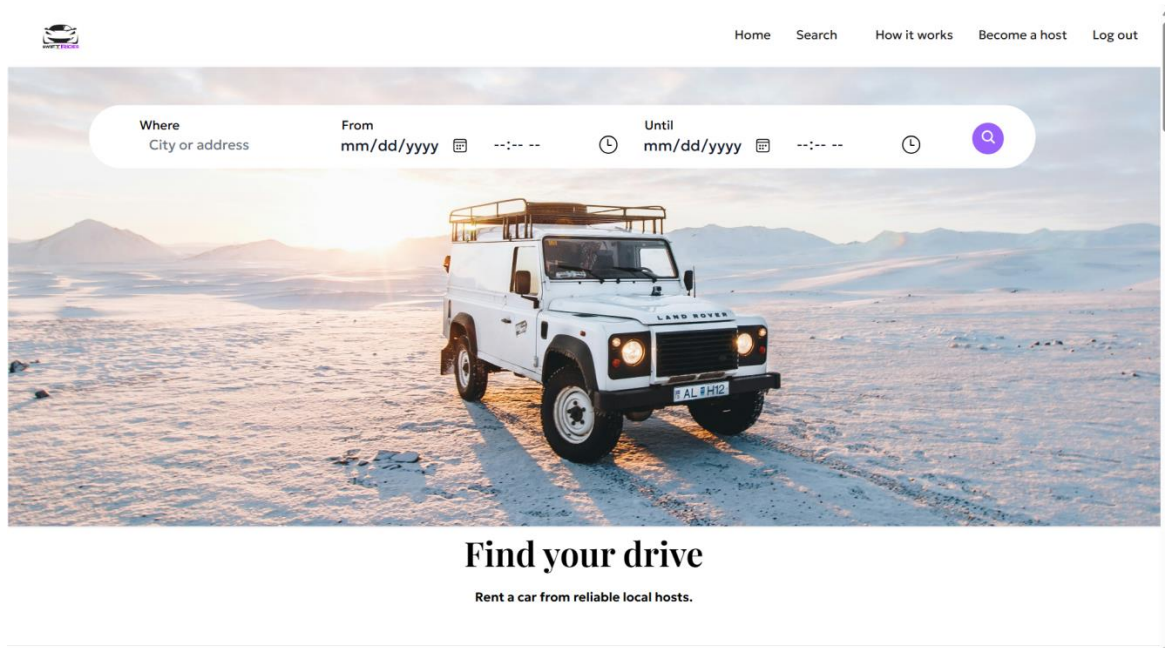
Welcome to ShiftRides

Sign up

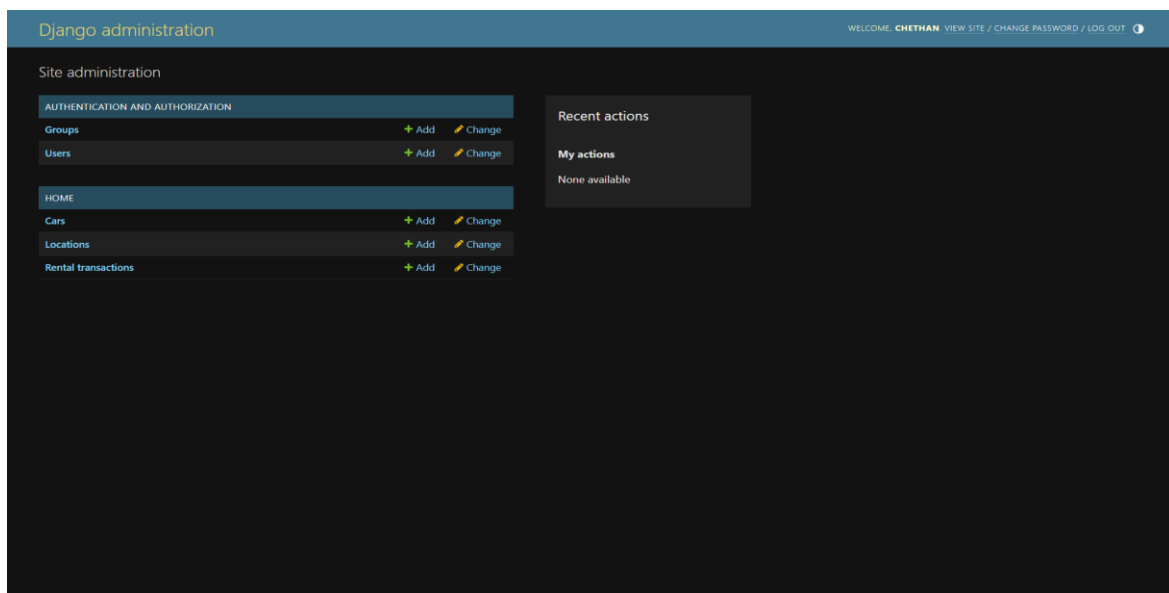
Already have an account?
[Login](#)

Sign up

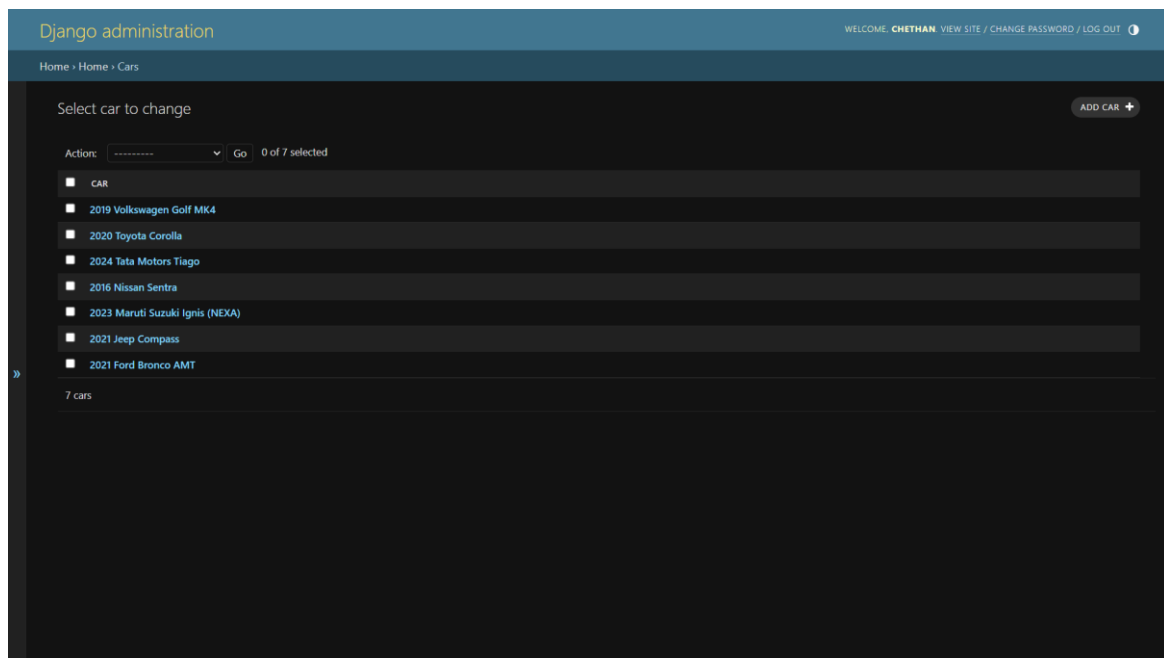
After Login page:



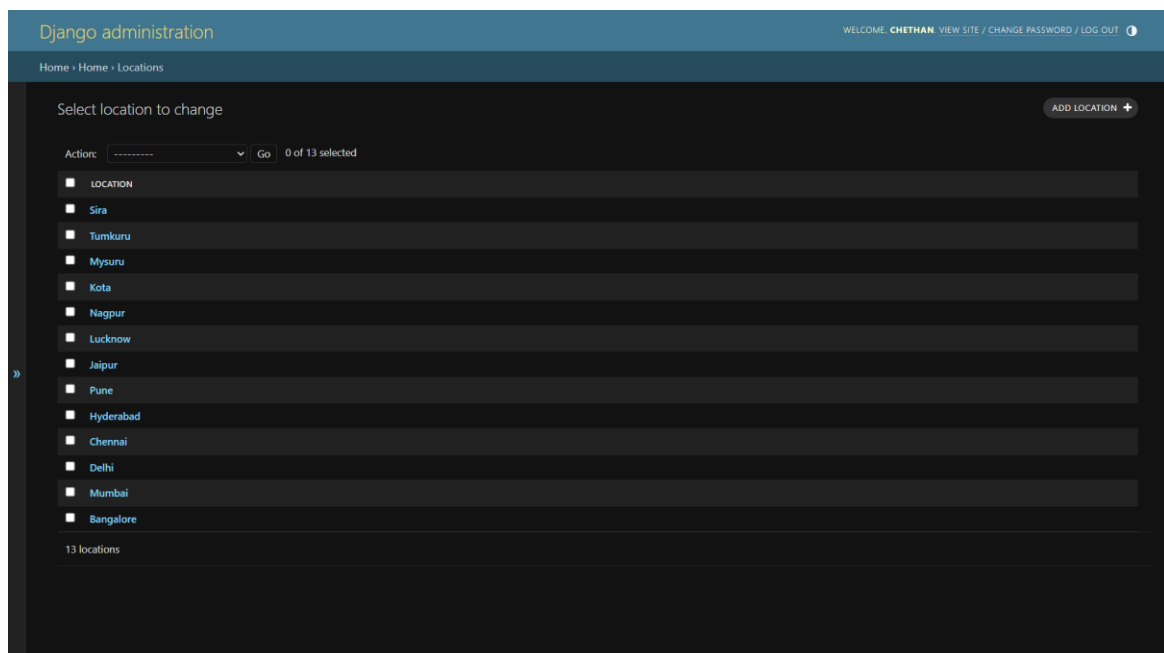
Admin Panel Overview page:



Car model view for Admin:



Location model view for Admin:



5.3 Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement. For a project like Swift Rides, which involves a complex web application with various user interactions and backend processes, **integration testing** would be one of the best testing methods to implement. Here's why:

Integration Testing

Integration testing focuses on testing how different modules or components of the system work together as a whole. In the context of Swift Rides:

- **System Integration:** Tests how various modules (such as user interface, database interactions) integrate and function together.
- **API Integration:** Ensures that APIs integrate correctly with the application and handle data exchange securely and efficiently.
- **Database Integration:** Verifies that data flows correctly between the application and databases (e.g., storing user profiles, managing vehicle inventory) without data loss or corruption.
- **User Interface Integration:** Checks that the front-end components interact correctly with backend services and data, ensuring a seamless user experience.
- **Comprehensive Coverage:** Tests the entire system and its interactions, identifying issues that may not be caught by unit tests alone.
- **Realistic Scenarios:** Mimics real-world scenarios, including how different parts of the application interact during user interactions and transactions.

- **Early Detection of Issues:** Helps detect integration issues early in the development cycle, reducing debugging time and costs.
- **Improved Reliability:** Ensures that the application functions correctly as a unified system, enhancing overall reliability and user satisfaction.

Implementation Tips

- **Automated Testing:** Use automated tools and scripts to perform integration tests regularly, ensuring consistent and thorough testing.
- **Mocking and Stubs:** Utilize mocks or stubs for external dependencies (like APIs) to simulate behaviors and control test scenarios effectively.
- **Test Environments:** Set up dedicated test environments that closely mirror the production environment to simulate real-world conditions accurately.

By implementing integration testing as a core testing method for Swift Rides, you can ensure that the entire application works seamlessly together, providing a robust and reliable experience for users.

Chapter - 6

Conclusion

- 6.1 Conclusion
- 6.2 Future Enhancement
- 6.3 Bibliography

6.1 Conclusion

Swift Rides represents a modern and user-centric approach to car rental services, leveraging advanced technologies to provide a seamless online platform. With a focus on intuitive user interfaces, robust backend systems, secure payment gateways, and comprehensive integration with third-party services, Swift Rides aims to enhance customer satisfaction and operational efficiency. By adopting agile development practices, continuous improvement through user feedback, and adherence to industry standards, Swift Rides is poised to deliver a reliable and scalable solution that meets the diverse needs of its users in the competitive car rental market.

6.2Future Enhancements

For future enhancements, We have plans on implementing a UPI gateway integrated with other payments like Debit/Credit cards, Net-banking and others A unified Payment Dashboard(under development). Customer Review / Feedback / Ratings Dashboard. Recommendation engine for greater usage of application. Implementation of AI, ML to expanding in larger communities.

6.3Bibliography

- www.github.com
- www.thecleverprogrammer.com
- www.researchgate.net
- www.geeksforgeeks.com
- <http://www.w3schools.com>